

NOCQ: A Constraint-Based Toolchain for Parity with Quantitative Conditions

Gonzalo Hernandez ✉

Monash University

Universidad de Nariño

Julian Garcia ✉

Monash University

Julian Gutierrez ✉

University of Sussex

Guido Tack ✉

Monash University

OPTIMA ARC ITTC

Abstract

Solving parity games is a cornerstone problem in formal verification and synthesis, with significant applications across industrial-scale system design. We present **NOCQ**, an efficient constraint-based toolchain designed to solve parity games under quantitative conditions. By leveraging the inherent duality of zero-sum game players, **NOCQ** implements several constraint programming (CP) techniques that exploit search space symmetries. Our experimental evaluation demonstrates that **NOCQ** performs competitively against state-of-the-art specialized solvers, often matching or exceeding their performance, while maintaining the modularity and flexibility of a CP-based framework. This flexibility allows users to easily incorporate additional constraints to search for preferred solutions, such as average quantitative performance requirements. We describe the tool's architecture, provide a comprehensive performance analysis, and discuss its extensibility. **NOCQ** is open-source and available at: [anonymous]

2012 ACM Subject Classification Theory of Computation → Constraint and Logic Programming; Theory of Computation → Algorithmic Game Theory; Theory of Computation → Verification by Model Checking

Keywords and phrases constraint programming, game theory, parity games, synthesis of reactive systems, model checking

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Acknowledgements I want to thank ...

1 Introduction

Parity games are two-player zero-sum infinite-duration games played on labelled directed graphs. Solving them is fundamental to automated formal verification, specifically model checking and synthesis. While the problem lies in $\text{NP} \cap \text{co-NP}$, a polynomial-time solution remains elusive despite decades of research. Existing algorithms often encounter exponential runtime limitations in practice. One promising approach involves reducing parity games to SAT instances, which ensures that a winning strategy exists for Player Even if and only if the instance is satisfiable. Building on the CP-based techniques introduced in [17], we present **NOCQ**: a platform that outperforms previous CP approaches by several orders of magnitude and competes effectively with state-of-the-art solvers [23, 3, 1]. **NOCQ** extends these algorithms to support quantitative restrictions, such as energy [7] and mean-payoff [8] constraints, within a unified SAT/CP-based framework. This tool paper describes the architecture,

 © Gonzalo Hernandez, Julian Garcia, Julian Gutierrez, , Guido Tack;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:10

 Leibniz International Proceedings in Informatics

Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

44 implementation, and usage of NOCQ, and provides experimental results demonstrating its
45 efficiency in solving games with these additional quantitative requirements.

46 **2 Preliminaries and Overview**

47 This section presents the graph games that NOCQ solves, and some translations into Boolean
48 Satisfiability (SAT) and Satisfiability Modulo Theories (SMT).

49 **2.0.1 Arenas, Strategies, and Plays.**

50 A game arena is a tuple $\mathcal{A} = (V_\circ, V_\square, E, \Omega, Q)$, where $V = V_\circ \cup V_\square$ is a finite set of vertices
51 partitioned between Player $\circ$ (0) and Player $\square$ (1), $E \subseteq V \times V$ is a set of directed edges
52 where every vertex has at least one successor, $\Omega : V \rightarrow \{0, \dots, d\}$ is a priority mapping, and
53 $Q : E \rightarrow \mathbb{Z}$ is a weight function. A *positional strategy* for player $i \in \{\circ, \square\}$ is a function
54 $f_i : V_i \rightarrow V$ such that $(v, f_i(v)) \in E$. A pair of strategies $(f_\circ, f_\square)$ uniquely determines
55 an infinite path $\pi = v_0 v_1 \dots$, called a *play*. We denote by $\text{Inf}(\pi)$ the set of priorities
56 $\{\Omega(v) \mid v \text{ occurs infinitely often in } \pi\}$.

57 **2.0.2 Winning Conditions.**

58 A play π is evaluated against three different criteria:

- 59 1. **Parity** (ω): Satisfied if $\max(\text{Inf}(\pi))$ is even.
- 60 2. **Energy** (η): For $E_{\text{init}} \in \mathbb{N}$, satisfied if $\forall n \geq 0 : E_{\text{init}} + \sum_{i=0}^{n-1} Q(v_i, v_{i+1}) \geq 0$.
- 61 3. **Mean-Payoff** (μ): Satisfied if $\liminf_{n \rightarrow \infty} \frac{1}{n} \sum_{i=0}^{n-1} Q(v_i, v_{i+1}) \geq 0$.

62 We define Player $\circ$'s winning goal as the combined condition formed by the conjunction
63 of these requirements: $\gamma_\circ = \omega \wedge \eta \wedge \mu$. As these are zero-sum games, Player $\square$ wins if the
64 play satisfies $\gamma_\square = \neg \gamma_\circ$. Given an initial vertex v_0 , a strategy is said to be winning from
65 v_0 for Player $\circ$ (Player $\square$) if, starting at v_0 , it only generates winning plays for Player $\circ$
66 (Player $\square$), for all strategies of the other player. It is known that for every vertex $v \in V$
67 in these games, one of the two players always has a winning strategy from that vertex v .
68 Solving a zero-sum game is deciding, for every $v \in V$, which player has a winning strategy
69 from that vertex.

70 **2.0.3 Solving Quantitative Parity Games.**

71 We solve quantitative parity games by encoding them into a Constraint Satisfaction Problem
72 (CSP) with a specialized propagator [17], as follows:

73 ► **Definition 1.** The CSP for an arena $\mathcal{A}$ starting at v_0 from the perspective of player $\mathbf{p}$ is
74 the triple $\mathcal{P} = (\mathcal{X}, \mathcal{D}, \mathcal{C})$, where $\mathcal{X} = \{\mathcal{V}_v\}_{v \in V} \cup \{\mathcal{E}_e\}_{e \in E}$ are Boolean variables $\mathcal{D}$, and the
75 constraints $\mathcal{C}$ are defined in the following way:

$$\begin{aligned}
 76 \quad & \mathcal{C}_{v_0} : \mathcal{V}_{v_0} = \top \\
 77 \quad & \mathcal{C}_{\mathbf{p}} : \bigwedge_{v \in V_{\mathbf{p}}} (\mathcal{V}_v \rightarrow (\sum_{w \in vE} \mathcal{E}_{(v,w)} = 1)) \\
 78 \quad & \mathcal{C}_{\bar{\mathbf{p}}} : \bigwedge_{v \in V_{\bar{\mathbf{p}}}} (\mathcal{V}_v \rightarrow \bigwedge_{w \in vE} \mathcal{E}_{(v,w)}) \\
 79 \quad & \mathcal{C}_E : \bigwedge_{w \in V, w \neq v_0} ((\bigvee_{v \in wE} \mathcal{E}_{(v,w)}) \rightarrow \mathcal{V}_w) \\
 80 \quad & \mathcal{C}_{\text{NOC}}(f) : \bigwedge_{v_0, \dots, v_k} \left(\left(\bigwedge_{i=0}^{k-1} \mathcal{E}_{(v_i, v_{i+1})} \right) \rightarrow \bigwedge_{j=0}^{k-1} ((v_j = v_k) \rightarrow f(v_j, \dots, v_k)) \right) \\
 81 \quad & f_\omega(v_j, \dots, v_k) = \max\{\Omega(v_i) \mid i \in j \dots k\} \bmod 2 \neq \bar{\mathbf{p}}
 \end{aligned}$$

$$f_\eta(v_j, \dots, v_k) = \sum_{i=j}^{k-1} Q(v_i, v_{i+1}) \geq 0$$

$$f_\mu(v_j, \dots, v_k) = \frac{1}{k-j} \sum_{i=j}^{k-1} Q(v_i, v_{i+1}) \geq \nu$$

$\mathcal{C}_{v_0}, \mathcal{C}_p, \mathcal{C}_{\bar{p}}$, and $\mathcal{C}_E$ enforce connectivity –ensuring valid moves for Player $\bigcirc$ and $\square$, respectively–, where p select exactly one successor and $\bar{p}$ should select all of them.

Moreover, $\mathcal{C}_{NOC}$ ensures that every reachable cycle is winning for p by satisfying the given combination of parity (ω), energy (η), and mean-payoff (μ) requirements. Lastly, in f_μ , parameter $\nu \in \mathbb{Z}$ is a given performance threshold used to evaluate the mean-payoff condition. This generalized construction allows NOCQ to search from the perspective of either player by swapping p and $\bar{p}$. Satisfiability under p proves a winning strategy for p , while unsatisfiability (if the solver explores the full space) proves the win for $\bar{p}$.

2.0.4 A No-Opponent-Cycle filter.

We have developed a complete propagator based on Depth-first Search (DFS) to explore the arena. The resulting DFS tree captures all possible paths from v_0 , and can be used to detect all existing cycles. We define the propagator in the context of a *Lazy Clause Generation* solver [12], to take advantage of the strong performance of this type of solver, in particular for problems that contain a very large number of Boolean clauses. Algorithm 1 presents this version of the propagator, which checks for cycles and reports failure if a cycle satisfying the winning condition for $\bar{p}$ is detected.

Algorithm 1 NOCFilter

Input: $path_V, path_E, v, e, \mathcal{E}$, defined

Output: $\top$ is valid; $\perp$ otherwise

```

1: if  $v \in path_V$  then
2:    $i_v \leftarrow index(v, path_V)$ 
3:   if  $\neg \gamma_p$  then
4:      $reason \leftarrow clausify(\neg path_E[i_v :])$ 
5:     if not  $(\mathcal{E}[e] \leftarrow (\perp, reason))$  then return  $\perp$ 
6:   else if defined then
7:     for  $e' \in edges(v)$  where  $\mathcal{E}[e'] \neq \perp$  do
8:        $p_V \leftarrow path_V \cup \{v\}$ 
9:        $p_E \leftarrow path_E \cup \{e'\}$ 
10:       $w \leftarrow target(e')$ 
11:       $def \leftarrow \mathcal{E}[e'] = \top$ 
12:       $NOCeager(p_V, p_E, w, e', \mathcal{E}, def)$ 
13: return  $\top$ 

```

Algorithm 1 implements the No-Opponent-Cycle propagator using a recursive DFS traversal. It maintains the current exploration state via $path_V$ and $path_E$. Parameter $\mathcal{E}$ tracks the Boolean decision variables for arena edges. The core logic occurs when a cycle is detected (Line 1). The propagator evaluates γ_p —the specific combination of parity (ω), energy (η), and mean-payoff (μ) objectives in the game. If the cycle violates the winning criteria for p , the propagator rules out the edge that closes the cycle (Line 5), explained by a *conflict clause* generated in Line 4 consisting of the negation of the edges currently in $path_E$. This clause effectively *learns* the failing partial assignment, allowing the underlying SAT/CP solver to perform non-chronological backtracking whenever this assignment is part of a failure and prune all other search branches that can lead to the same non-winning cycle.

3 Tool Description

In this section, we describe the implementation of NOCQ, a C++ framework developed to model and solve parity games with quantitative conditions. Unlike *Oink* [23], which focuses on explicit-state systems and symbolic algorithms, NOCQ is designed to leverage Constraint Programming (CP) and SAT technologies. The source code, implementation details, usage instructions, and a number of examples are available at [anonymous]

Core Architecture and Modules

NOCQ is organised into four independent modules: *Game Manager*, *CPSModels*, *SATEncoding*, and *Utils*. This modular design ensures flexibility, allowing the tool to be used either as a standalone CP/SAT solver or as a reusable library.

Game Manager.

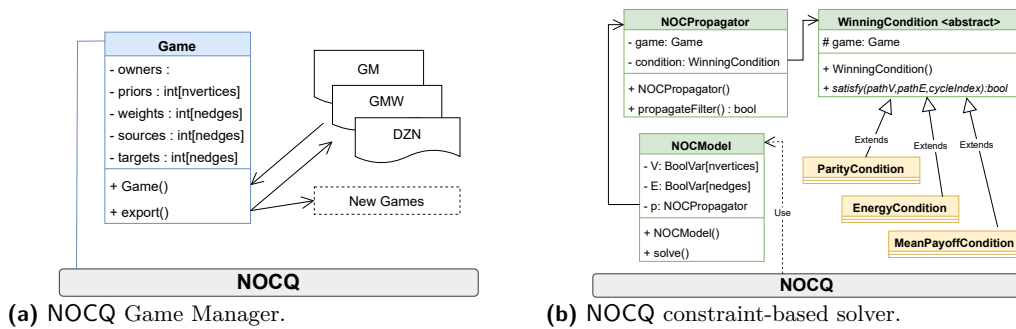
As shown in Figure 1a, the **Game** class serves as the core data structure of NOCQ. It captures essential game attributes, including vertex ownership (**owners**), priorities (**priors**), and edge weights (**weights**). In addition, it maintains the arena topology through **sources** and **targets** arrays for all edges.

3.0.0.1 Input and Procedural Generation.

The framework supports standard formats used to represent games, such as GM [19], PG [23], and DZN [21]. We also define the GMW (**.gmw**) format, an extension of GM specifically designed to store edge weights. Moreover, following Parys [19], **Game** supports the procedural generation of *Random*, *Jurdziński*, and *MLadder* games for benchmarking purposes.

Users can interact with the Game Manager either to load existing benchmarks or to generate new game instances. The first command below loads a parity game and assigns random edge weights in the range $[-20, 20]$; by default, the solver maximizes for the parity reward. The second line generates a Jurdziński game with 15,000 levels and 50 blocks, set to minimize the reward:

```
$ ./nocq --gm /pgsolver/randomgame-20000-10-1-20-0.gm --weights -20 20
$ ./nocq --jurd 15000 50 --weights -50 30 --min
```



■ **Figure 1** Overview of the main NOCQ Toolkit Components.

Constraint-Based Solving (CPSModels).

Figure 1b illustrates how NOCQ implements the CSP model in Definition 1 and integrates the No-Opponent-Cycle propagators described in Section 2.0.4. A key design principle of NOCQ is model independence: the underlying model can be interfaced with different constraint programming (CP) engines. Once a game is loaded or created, an instance of `NOCModel` is instantiated and used to solve the problem. The `NOCModel` searches for solutions by enforcing all constraints, including C_{NOC} from Definition 1. This constraint is implemented through `NOCPropagator`, which uses CP techniques to evaluate winning conditions over all relevant cycles in the input game.

In the current version, three different winning conditions are implemented. The system is extensible to incorporate new winning conditions by extending the `WinningCondition` class with additional objectives. The framework has been validated using two state-of-the-art solvers:

- **Chuffed** [12], a lazy clause generation solver.
- **Gecode** [20], a comprehensive constraint-based toolkit.

Once a game has been loaded or created, one can invoke the constraint-based solver from the command line. Several parameters are available to tailor the solving process. The following command solves a parity+energy game (threshold=20) from a given initial vertex (18), and reports the execution times:

```
$ ./nocq --gm /games/rand_50_20_1_5_-20_30.gmw --noc --init 18
--parity --energy 20 --print-times
```

The architecture of NOCQ is very modular, allowing new CP solvers to be integrated into the whole toolchain using the model developed in [17].

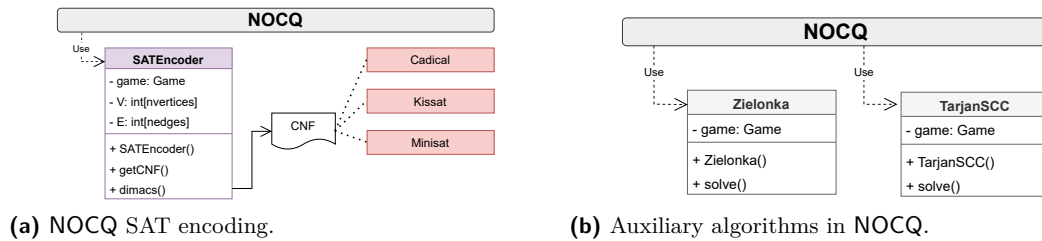


Figure 2 Overview of additional components of NOCQ.

SAT Encoding.

NOCQ also provides a dedicated SAT encoding pipeline, enabling parity games to be dispatched to a number of high-performance SAT solvers, including *CaDiCaL* [4], *Kissat* [4], and *MiniSat* [14]. Figure 2a illustrates the `SATEncoder` class, which implements the encoding techniques proposed by Heljanko *et al* [16].

The `SATEncoder` maintains references to the `Game` instance and maps graph components to integer representations of vertices (V) and edges (E). The module produces standard CNF output in DIMACS format, ensuring compatibility with any compliant SAT solver. We note that the SAT reduction proposed in [16] is restricted to parity objectives/games only, and does not support quantitative winning conditions such as energy or mean-payoff, as we do within NOCQ.

The following example encodes the `towersofhanoi-5` game from the `pgsolver` collection [15], starting from vertex 15, into CNF format. The encoding targets reward minimization, as described in [16]. The resulting output, a DIMACS file, can then be passed onto any SAT solver, such as `CaDiCaL`:

```
$ ./nocq --gm /pgsolver/towersofhanoi-5.gm --sat-encoding tower5.cnf --init 15
$ ./cadical tower5.cnf
```

Auxiliary Algorithms (Utils).

Beyond CP and SAT technologies, NOCQ includes procedures for evaluation and preprocessing, as illustrated in Figure 2b:

- **Zielonka** [1]: parity games algorithm with improved attractor computation.
- **TarjanSCC** [22]: efficient SCC decomposition of game arenas.

The command below generates a random game (20 vertices, 10 priorities, degree 1–5), solves it using the Zielonka Recursive Algorithm (ZRA), and exports the arena to `rand.gm`, subsequently performing SCC decomposition:

```
$ ./nocq --rand 20 10 1 5 --zra --print-solution --export-gm rand.gm
$ ./nocq --gm rand.gm --scc
```

NOCQ users can view the full list of supported parameters using `./nocq -help`. A more in-depth explanation of specific configurations and advanced usage is available as part of the documentation in the Git repository of NOCQ.

3.0.1 Exploiting Problem Duality through Parallel Solving.

The performance of game solvers is strongly influenced by the topological and quantitative structure of the arena. A key observation is the *asymmetry in computational cost*: game instances that are difficult to solve from the perspective of player p are often easier (in practical terms) from the perspective of $\bar{p}$, and vice versa. To exploit this duality, NOCQ executes the CSP concurrently from both perspectives in separate threads. The computation terminates as soon as one thread identifies a winning strategy or proves its absence. This parallel approach mitigates worst-case behavior by leveraging the more computationally favorable viewpoint, which depends on the particular game that is given to NOCQ as input.

4 Experiments and Evaluation

In this section, we evaluate NOCQ across two dimensions: qualitative parity games and quantitative multi-objective verification. All experiments were conducted on an Intel Xeon 8260 CPU with 24 cores (non-hyperthreaded) and 268.55 GB of RAM running Linux.

4.1 Parity conditions

First, we assess its performance on games featuring parity conditions only, comparing NOCQ against state-of-the-art algorithms using standard benchmarks.

For the SAT backends, we employ several high-performance solvers: **CaDiCaL** (v2.1.3) [4], **MiniSat** (v2.2.0) [14], and **Kissat** (v4.0.2) [4]. To establish a robust baseline, we compare our results with **Oink** [23], which implements state-of-the-art algorithms such as

Priority Promotion (PP and PP+) and Parys' improvements (PAR). Additionally, we include Zielonka's Recursive Algorithm (ZRA) featuring the optimized attractor strategy presented in [1], which is not yet integrated into Oink.

The benchmarks used were taken from the PGSolver Library [15] and Keiren's benchmarks for parity games [18]. The evaluation utilizes 469 benchmark instances categorized into four groups: **Equivalence Checking (EqC)**, containing 77 instances from formal verification tasks with complex structural connectivity; **Model Checking (MoC)**, consisting of 106 organic benchmarks derived from μ -calculus problems; the **PGSolver Collection (PGS)**, with 161 games from difficult families designed to test theoretical worst-case bounds; and **Random Games (RaG)**, comprising 125 instances with varied priority distributions used to assess average-case efficiency and general robustness.

We focus our evaluation on large-scale benchmarks, which provide a more rigorous test for the scalability of NOCQ against dedicated solvers. This category includes arenas with up to 35 million vertices, representing real-world verification challenges where traditional SAT-based methods often fail to scale. By prioritizing these high-complexity instances, we demonstrate NOCQ's robustness in industrial-scale formal verification tasks.

■ **Table 1** Performance of algorithms on large-scale instances. Average time is reported in seconds using PAR2 scoring with a 60s timeout.

Algorithm	EquivalenceCh		ModelCh		PGSolver		RandomG	
	Time	Solved	Time	Solved	Time	Solved	Time	Solved
Oink(PP)	24.775	44/44	9.547	49/49	15.204	88/93	0.016	100/100
Oink(PP+)	24.876	44/44	9.557	49/49	15.204	88/93	0.015	100/100
Oink(PAR)	24.159	44/44	8.718	49/49	20.690	85/93	0.015	100/100
ZRA(ImpAttr)	14.490	44/44	3.929	49/49	21.804	85/93	0.053	100/100
NOCQ (Ours)	13.502	44/44	3.837	49/49	9.206	92/93	0.035	100/100
SAT(CaDiCaL)	—	—	—	—	—	—	57.509	14/25
SAT(Kissat)	—	—	—	—	—	—	69.136	12/25
SAT(MiniSat)	—	—	—	—	—	—	58.297	14/25

As shown in Table 1, standard SAT-based approaches are omitted for several categories due to their inability to scale within reasonable resource limits. In this challenging context, NOCQ demonstrates superior robustness and scalability. Specifically, it achieves the best performance on Equivalence Checking and Model Checking benchmarks. Furthermore, on the harder problem families within the PGSolver collection, NOCQ solves the highest number of instances (92/93) and is significantly faster. While specialized solvers remain slightly faster on Random Games, NOCQ remains competitive within a factor of two, highlighting its effectiveness as a general-purpose solver for large-scale parity games.

4.2 Quantitative Goals and Verification

One of NOCQ's main features is the ability to handle quantitative goals within the same CP-based framework. NOCQ implements a core algorithm designed to compute positional strategies, which are optimal in case only qualitative (parity) or only quantitative goals are given. However, when combined, positional strategies may no longer be optimal [10, 6]. As such, the solutions that NOCQ computes are for games where only positional strategies are allowed. In this context, the next set of benchmarks demonstrates NOCQ's performance when solving such kinds of games when enriched with additional quantitative (energy and/or mean-payoff) requirements. To this end, we solve a selection of random benchmarks as proposed

in [5], following the design of the SPRAND generator [11] for energy and mean-payoff games. These benchmarks utilize random positional vertex assignments and randomly generated thresholds. We assess the tool’s efficiency across various multi-objective configurations defined by the winning condition $\gamma_p = \omega \otimes \eta \otimes \mu$, where $\otimes$ denotes the product composition of parity (ω), energy (η), and mean-payoff (μ) objectives.

The benchmarks are categorized by their structural properties using the naming convention $R_{\langle d \rangle \langle s_key \rangle}$, where $\langle d \rangle$ represents the out-degree (density) and $\langle s_key \rangle$ denotes the state-space size. We consider two density levels: low density (5) and high density (10). The vertex counts are scaled as follows: $m = 2^{16}$, $b = 2^{17}$, $h = 2^{18}$, $xb = 2^{19}$, and $xh = 2^{20}$.

Table 2 Performance of NOCQ with different winning conditions. Average time is reported in seconds using PAR2 scoring with a 60s timeout.

Conditions	R_(5,10)m		R_(5,10)b		R_(5,10)h		R_(5,10)xb		R_(5,10)xh	
	Time	Solved	Time	Solved	Time	Solved	Time	Solved	Time	Solved
ω	0.067	100/100	0.365	100/100	0.379	100/100	1.369	100/100	1.788	100/100
η	0.065	100/100	0.462	100/100	17.037	87/100	48.483	60/100	38.388	69/100
μ	0.065	100/100	6.342	95/100	13.504	89/100	10.313	92/100	30.084	76/100
$\omega \otimes \eta$	0.065	100/100	1.605	99/100	2.754	99/100	1.978	99/100	1.773	100/100
$\omega \otimes \mu$	0.066	100/100	6.339	95/100	13.507	89/100	10.318	92/100	30.064	76/100
$\eta \otimes \mu$	0.066	100/100	6.346	95/100	13.505	89/100	11.504	91/100	30.070	76/100
$\omega \otimes \eta \otimes \mu$	0.066	100/100	6.347	95/100	13.507	89/100	11.497	91/100	30.047	76/100

The results across Table 2 reveal that the performance of NOCQ is highly dependent on the specific combination of objectives and the density of the game graph. For single-objective instances, the parity condition (ω) remains consistently the easiest to solve, maintaining a 100% success rate regardless of graph size or density. In contrast, energy (η) and mean-payoff (μ) objectives exhibit greater sensitivity to state-space scaling; as the vertex count increases, there is a noticeable decline in the number of instances solved within the time limit.

A key strength of the NOCQ toolchain is highlighted in the multi-objective configurations. When combining parity with an energy objective (e.g., $\omega \otimes \eta$), the solver frequently demonstrates a pruning effect, where the parity condition helps narrow the search space, leading to more robust solving times compared to other combinations. However, for the most complex triple-objective configurations ($\omega \otimes \eta \otimes \mu$), the performance tends to be bottlenecked by the mean-payoff component.

Since the benchmarking software in [5] has not been released, we could not perform a side-by-side comparison. Nevertheless, our results for energy games show a reasonable solving time compared with their reported results. Furthermore, obtaining standardized benchmarks for this class of games remains a challenge, as few publicly available datasets or implementations exist.

5 Conclusions and Related Work

As demonstrated by our experimental evaluation, NOCQ is an efficient toolchain that leverages a constraint-based approach to solve games on graphs with performance competitive with state-of-the-art algorithms used in formal verification to solve parity games. A key advantage of NOCQ lies in its ability to solve parity games combined with energy and mean-payoff conditions simultaneously, maintaining high performance without increasing implementation complexity. Because NOCQ is built upon CP, the model is inherently extensible; new constraints can be added seamlessly without modifying the underlying solver architecture.

Finally, several tools have been developed for quantitative games only – *e.g.*, see [2] and references therein. But, much less work has been done regarding practical tools for parity games (or ω -regular games more generally) with additional quantitative requirements. In [9, 13] two attempts were made, but in such cases full parity games with quantitative objectives were not evaluated in practice. Our results provide a step forward where the case for positional strategies is accounted for in complex parity games with additional quantitative requirements.

References

- 1 Saksham Aggarwal, Alejandro Stuckey de la Banda, Luke Yang, and Julian Gutierrez. A matrix-based approach to parity games. In Sriram Sankaranarayanan and Natasha Sharygina, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 666–683, Cham, 2023. Springer Nature Switzerland.
- 2 Massimo Benerecetti, Daniele Dell’Erba, and Fabio Mogavero. Solving mean-payoff games via quasi dominions. *Inf. Comput.*, 297:105151, 2024. URL: <https://doi.org/10.1016/j.ic.2024.105151>, doi:10.1016/J.IC.2024.105151.
- 3 Massimo Benerecetti, Daniele Dell’Erba, Fabio Mogavero, Sven Schewe, and Dominik Wojtczak. Priority promotion with parysian flair. *Journal of Computer and System Sciences*, 147:103580, 2025. URL: <https://www.sciencedirect.com/science/article/pii/S0022000024000758>, doi:10.1016/j.jcss.2024.103580.
- 4 Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024. In Marijn Heule, Markus Iser, Matti Järvisalo, and Martin Suda, editors, *Proceedings of SAT Competition 2024: Solver, Benchmark and Proof Checker Descriptions*, volume B-2024-1 of *Department of Computer Science Report Series B*, pages 8–10, University of Helsinki, Finland, 2024. URL: <https://cca.informatik.uni-freiburg.de/papers/BiereFallerFazekasFleuryFroleyksPollitt-SAT-Competition-2024-solvers.pdf>.
- 5 Jakub Chaloupka. *Algorithms for Mean-Payoff and Energy Games*. PhD thesis, Masaryk University, Faculty of Informatics, Brno, Czech Republic, February 2011. URL: https://is.muni.cz/th/bvfcf/phd_thesis.pdf.
- 6 Krishnendu Chatterjee and Laurent Doyen. Energy parity games. *CoRR*, abs/1001.5183, 2010. URL: <http://arxiv.org/abs/1001.5183>, arXiv:1001.5183.
- 7 Krishnendu Chatterjee and Laurent Doyen. Energy parity games. *Theor. Comput. Sci.*, 458:49–60, 2012. URL: <https://doi.org/10.1016/j.tcs.2012.07.038>, doi:10.1016/J.TCS.2012.07.038.
- 8 Krishnendu Chatterjee, Monika Henzinger, and Alexander Svozil. Faster algorithms for mean-payoff parity games. In Kim G. Larsen, Hans L. Bodlaender, and Jean-François Raskin, editors, *42nd International Symposium on Mathematical Foundations of Computer Science, MFCS 2017, Aalborg, Denmark, August 21-25, 2017*, volume 83 of *LIPIcs*, pages 39:1–39:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. URL: <https://doi.org/10.4230/LIPIcs.MFCS.2017.39>, doi:10.4230/LIPIcs.MFCS.2017.39.
- 9 Krishnendu Chatterjee, Thomas A. Henzinger, Barbara Jobstmann, and Rohit Singh. QUASY: quantitative synthesis tool. In Parosh Aziz Abdulla and K. Rustan M. Leino, editors, *Proc. 17th Int. Conf. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 6605 of *Lecture Notes in Computer Science*, pages 267–271. Springer, 2011. doi:10.1007/978-3-642-19835-9_24.
- 10 Krishnendu Chatterjee, Thomas A. Henzinger, and Marcin Jurdzinski. Mean-payoff parity games. In *20th IEEE Symposium on Logic in Computer Science (LICS 2005), 26-29 June 2005, Chicago, IL, USA, Proceedings*, pages 178–187. IEEE Computer Society, 2005. doi:10.1109/LICS.2005.26.

- 333 11 Boris V. Cherkassky, Andrew V. Goldberg, and Tomasz Radzik. Shortest paths algorithms:
334 Theory and experimental evaluation. *Mathematical programming*, 73(2):129–174, 1996.
- 335 12 Geoffrey Chu and Peter J. Stuckey. Chuffed: A lazy clause generation constraint solver.
336 GitHub repository, 2018. URL: <https://github.com/chuffed/chuffed>.
- 337 13 Sven Dziadek, Uli Fahrenberg, and Philipp Schlehuber. ω -regular energy problems. *Formal*
338 *Aspects Comput.*, 37(1):4:1–4:30, 2025. doi:10.1145/3678265.
- 339 14 Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In Enrico Giunchiglia and
340 Armando Tacchella, editors, *Theory and Applications of Satisfiability Testing (SAT 2003)*,
341 volume 2919 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2003. doi:
342 10.1007/978-3-540-24605-3_37.
- 343 15 Oliver Friedmann and Martin Lange. *The PGSolver Collection of Parity Game Solvers*,
344 *Version 4.1*, June 2017. Accessed: 2025-03-13. URL: [https://github.com/tcsprojects/](https://github.com/tcsprojects/pgsolver/blob/master/doc/pgsolver.pdf)
345 [pgsolver/blob/master/doc/pgsolver.pdf](https://github.com/tcsprojects/pgsolver/blob/master/doc/pgsolver.pdf).
- 346 16 Keijo Heljanko, Misa Keinänen, Martin Lange, and Ilkka Niemelä. Solving parity games
347 by a reduction to SAT. *Journal of Computer and System Sciences*, 78(2):430–440,
348 2012. Games in Verification. URL: [https://www.sciencedirect.com/science/article/](https://www.sciencedirect.com/science/article/pii/S0022000011000535)
349 [pii/S0022000011000535](https://www.sciencedirect.com/science/article/pii/S0022000011000535), doi:10.1016/j.jcss.2011.05.004.
- 350 17 Gonzalo Hernandez, Julian Garcia, Julian Gutierrez, and Guido Tack. No-opponent-cycle
351 propagators for solving parity games. In *Integration of Constraint Programming, Artificial*
352 *Intelligence, and Operations Research (CPAIOR 2026)*, 2026. To appear.
- 353 18 Jeroen J. A. Keiren. Benchmarks for parity games (extended version), 2015. URL: [https:](https://arxiv.org/abs/1407.3121)
354 [//arxiv.org/abs/1407.3121](https://arxiv.org/abs/1407.3121), arXiv:1407.3121.
- 355 19 Paweł Parys. Parity games: Zielonka’s algorithm in quasi-polynomial time. *arXiv.org*, 2019.
- 356 20 Shulte, C., Tack, G., Lagerkvist, M. Modeling and programming with gecode, 2019. URL:
357 <https://www.gecode.dev/doc-latest/MPG.pdf>.
- 358 21 Peter J. Stuckey, Marriott Kim, and Tack Guido. The minizinc handbook, 2020. URL:
359 <https://www.minizinc.org/doc-latest/en/index.html>.
- 360 22 Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM journal on computing*,
361 1(2):146–160, 1972.
- 362 23 Tom van Dijk. Oink: An implementation and evaluation of modern parity game solvers. In
363 Dirk Beyer and Marieke Huisman, editors, *Tools and Algorithms for the Construction and*
364 *Analysis of Systems*, pages 291–308, Cham, 2018. Springer International Publishing.